

Дискретный анализ

Лекция 1: сортировки

Андрей Калинин

Дискретный анализ, 2026/27

12 сентября 2026

Что сегодня

- | | |
|--|----------|
| 1. Важность сортировки. | ≈ 20 мин |
| 2. Устройство <code>std::sort</code> и опыт Александреску. | ≈ 60 мин |
| 3. Чем больше известно о ключах, тем быстрее. | ≈ 60 мин |
| 4. Соревнование. | ≈ 10 мин |

Весь день — про одну задачу. Это не потому, что она лёгкая.

Самая проработанная задача в информатике

- У Кнута сортировке и поиску отведён целый третий том.
- «Arguably the most researched CS problem»; «every programmer must implement sort» — и почти каждый реализует её не сам.
- Задача решена — и всё равно каждый год выходят новые работы.

слайды Александреску, слайды 6–7; Кнут, т. 3, введение к гл. 5.

Почему в библиотеках именно quicksort

- Прост: разбиение — десяток строк.
- Быстр в среднем и с маленькой константой.
- Дёшев на почти отсортированных данных.
- Дружит с кэшем: работает подряд идущими кусками памяти.
- Сортирует на месте — лишней памяти $O(\log n)$ на стек.

Сортировка внутри других алгоритмов

- **Бинарный поиск** требует отсортированного массива — это следующее занятие.
- **Суффиксный массив** — буквально «отсортировать все суффиксы строки»: $A(S) = \arg \text{sort all_suffixes}(S)$. Тема «строки» позже в семестре.
- **Слияние отсортированных серий** — внешняя сортировка (сегодня, часть 3) и сортировка в СУБД: DuckDB сортирует поразрядной в каждом потоке, а потом сливает k серий.

Петабайт (Google, 2008)

- 1 ПБ = 10^{13} записей по 100 байт.
- 4000 серверов, 48 000 дисков, трёхкратное дублирование.
- Время: **6 ч 02 мин.** 1 ТБ — за 68 с на 1000 машин.
- При каждом запуске отказывает хотя бы один диск.

Блог Google, 21.11.2008, «Sorting 1PB with MapReduce» (сверено 10.09.2026). Старый кадр курса: слайды курса 2011 г..

Что происходит сейчас: Sort Benchmark

Номинация	Рекорд	Год
GraySort Indy	100 ТБ за 98,8 с на 512 узлах (Tencent Sort)	2016
GraySort Daytona	100 ТБ за 134 с	2016
CloudSort Indy	100 ТБ за \$97 (Exoshuffle-CloudSort)	2022
JouleSort	59,3 кДж; 169 тыс. записей/Дж (MendSort)	2023

sortbenchmark.org, «Top Results», проверено 11.09.2026.

Единица стоимости — не всегда сравнение

- В учебнике считают сравнения и перемещения.
- В Google 2008 считали обращения к диску и отказы оборудования.
- В JouleSort считают джоули.
- Сегодня мы дважды увидим, что и на одной машине «меньше сравнений» не значит «быстрее».

К модели стоимости вернёмся в части 3, во внешней сортировке.

Сортировка внутри LLM: top-p sampling

- Модель на каждом шаге выдаёт оценку («логит») каждому слову словаря.
- Тор-р: отсортировать словарь по оценке, накопить вероятность, отрезать хвост, выбирать только из головы.
- Значит, на *каждом сгенерированном токене* сортируется вектор размером со словарь.

HuggingFace transformers, src/transformers/generation/logits_process.py.

... и это буквально вызов сортировки

```
528 sorted_logits, sorted_indices = torch.sort(scores, descending=False)
529 cumulative_probs = sorted_logits.softmax(dim=-1).cumsum(dim=-1)
```

- class TopPLogitsWarper — строка 473, __call__ — 527, torch.sort — 528.

ветка main, коммит файла 9451e3ab (04.08.2026), прочитано 11.09.2026.

- C4 — Colossal Clean Crawled Corpus, очищенный слепок Common Crawl (апрель 2019), на котором учили T5; английская часть — около **750 ГБ** текста.
- Весь датасет склеивают в одну строку и строят суффиксный массив (8 байт на токен).
- Повторы длиной ≥ 50 токенов оказываются *соседями* в отсортированном массиве — их видно за один проход.
- Из C4 так удалено **7,18 %** токенов; одно предложение из 61 слова встречалось **61 036** раз.
- Модели на дедуплицированных данных воспроизводят заученный текст в **10 раз** реже.

Lee et al., «Deduplicating Training Data Makes Language Models Better», ACL 2022, arXiv:2107.06499v2, §§ 1, 4.1.1, 5.1; [E11]

Raffel et al., arXiv:1910.10683, § 2.2 (объём C4).

- Суффиксный массив в той работе строят алгоритмом SA-IS за линейное время: сравнений ключей там нет вообще.
- Это возможно, потому что про ключи известно больше, чем «их можно сравнить»: это суффиксы одной строки над конечным алфавитом.
- Ровно об этом — третья часть сегодняшнего дня.

Теперь — вниз, в реализацию

Что на самом деле делает `std::sort(v.begin(), v.end())`?

- Introsort: quicksort, который при глубокой рекурсии переключается на heapsort, а короткие куски подчищает сортировкой вставками.
- Дальше — *настоящий* код `libstdc++`, а не учебный.

STL = /usr/include/c++/13/bits/stl_algo.h, g++ 13.3; копия строк — demo/libstdcxx-introsort.md.

Как читать код стандартной библиотеки

- Имена с подчёркиваниями (`__sort`, `_S_threshold`) — внутренние имена реализации: стандарт резервирует их за библиотекой, чтобы не столкнуться с именами и макросами пользователя. Снаружи — `std::sort`.
- `_RandomAccessIterator` — параметр шаблона; читайте его как «указатель».
- `_GLIBCXX_MOVE(x)` — перемещение; для нас это обычное присваивание.
- `__comp` — компаратор, по умолчанию «меньше».
- Длинная сигнатура — это обобщённость: один и тот же код для любого типа, контейнера и компаратора. Алгоритмической информации в ней нет.
- Все сегодняшние выводы — про тела циклов: сколько сравнений и переходов, предсказуем ли переход.

1. Публичная обёртка

```
4851 sort(_RandomAccessIterator __first, _RandomAccessIterator __last)
4852 {
4853     // concept requirements
4854     __glibcxx_function_requires(_Mutable_RandomAccessIteratorConcept<
4855         _RandomAccessIterator>)
4856     __glibcxx_function_requires(_LessThanComparableConcept<
4857         typename iterator_traits<_RandomAccessIterator>::value_type>)
4858     __glibcxx_requires_valid_range(__first, __last);
4859     __glibcxx_requires_irreflexive(__first, __last);
4860
4861     std::_sort(__first, __last, __gnu_cxx::__ops::__iter_less_iter());
4862 }
```

- Вся видимая функция — проверки концептов и один вызов.

stl_algo.h, строки 4851-4862.

2. __sort: два шага и лимит глубины

```
1942 __sort(_RandomAccessIterator __first, _RandomAccessIterator __last,
1943         _Compare __comp)
1944 {
1945     if (__first != __last)
1946     {
1947         std::__introsort_loop(__first, __last,
1948                               std::__lg(__last - __first) * 2,
1949                               __comp);
1950         std::__final_insertion_sort(__first, __last, __comp);
1951     }
1952 }
```

- Два шага: `__introsort_loop` с лимитом глубины $2\lfloor \log_2 n \rfloor$ (при $n = 10^6$ — **38** уровней) и *один* проход вставками по всему массиву.
- Куски длиной ≤ 16 не сортируются вовсе: после цикла ни один элемент не дальше 16 позиций от места, и общий проход вставками дочищает это дешевле.

3. `__introsort_loop`: цикл вместо двух рекурсий

```
1922 while (__last - __first > int(_S_threshold))
1923 {
1924     if (__depth_limit == 0)
1925     {
1926         std::__partial_sort(__first, __last, __last, __comp);
1927         return;
1928     }
1929     --__depth_limit;
1930     _RandomAccessIterator __cut =
1931         std::__unguarded_partition_pivot(__first, __last, __comp);
1932     std::__introsort_loop(__cut, __last, __depth_limit, __comp);
1933     __last = __cut;
1934 }
```

`stl_algo.h`, строки 1922-1934.

... И ЧТО В НЁМ НАПИСАНО

- `__unguarded_partition_pivot` возвращает точку `__cut`: после него всё слева от неё не больше всего, что справа.
- Правая часть уходит в рекурсивный вызов. Левая — не рекурсия, а `__last = __cut` и следующий виток того же `while`: это ручное устранение хвостовой рекурсии, глубина стека растёт только на правых вызовах.
- Кончился лимит глубины — `__partial_sort`, то есть `heapsort`. Единственное место, где он виден в коде: страховка, а не рабочий режим.
- Куски короче порога не сортируются вовсе — цикл просто заканчивается, их дочищает финальный проход.

4. Опорный элемент: медиана трёх

```
1893 __unguarded_partition_pivot(_RandomAccessIterator __first,
1894                             _RandomAccessIterator __last, _Compare __comp)
1895 {
1896     _RandomAccessIterator __mid = __first + (__last - __first) / 2;
1897     std::__move_median_to_first(__first, __first + 1, __mid, __last - 1,
1898                               __comp);
1899     return std::__unguarded_partition(__first + 1, __last, __first, __comp);
1900 }
```

- Медиана `first+1`, `mid`, `last-1` переносится в `*first` и дальше служит опорным.
- Тело `__move_median_to_first` — STL:85-103, шесть случаев из двух-трёх сравнений.

`stl_algo.h`, строки 1893-1900.

5. Разбиение Хоара без проверок границ

Два указателя идут навстречу: левый стоит, пока элемент меньше опорного, правый — пока больше; оба встали — обмен; встретились — готово. Проверок границ нет: сторожем работает сам опорный.

```
1875 while (true)
1876 {
1877     while (__comp(__first, __pivot))
1878         ++__first;
1879     --__last;
1880     while (__comp(__pivot, __last))
1881         --__last;
1882     if (!(__first < __last))
1883         return __first;
1884     std::iter_swap(__first, __last);
1885     ++__first;
1886 }
```

У Ломута указатель один и опорный в конце: обменов заметно больше, зато опорный сразу встаёт на своё окончательное место.

6. Порог 16 и один финальный проход вставками

```
1848     enum { _S_threshold = 16 };
1856     {
1857         if (__last - __first > int(_S_threshold))
1858         {
1859             std::__insertion_sort(__first, __first + int(_S_threshold), __comp);
1860             std::__unguarded_insertion_sort(__first + int(_S_threshold), __last,
1861                                             __comp);
1862         }
1863         else
1864             std::__insertion_sort(__first, __last, __comp);
1865     }
```

Нижний фрагмент — целиком тело `__final_insertion_sort`, верхняя строка — константа порога, на которую оно ссылается. Эта функция вызывается *один раз*, после `__introsort_loop`.

`std_algo.h`, строки 1848 и 1856–1865.

Почему проверка нужна только первым 16

- Первые 16 элементов сортируются обычными вставками — с проверкой границы.
- После этого в `a[0]` лежит минимум *всего* массива: он обязан был оказаться среди первых шестнадцати, потому что после разбиений ни один элемент не дальше 16 позиций от своего места.
- Поэтому весь остаток дочищается без проверки границы: сторожем работает `a[0]`.
- Ветка `else` — когда весь массив короче 17 элементов: тогда обычные вставки по всему массиву, и никаких хитростей.

7. Внутренний цикл вставки — без $j > 0$

```
1794 {
1795     typename iterator_traits<_RandomAccessIterator>::value_type
1796     __val = _GLIBCXX_MOVE(*__last);
1797     _RandomAccessIterator __next = __last;
1798     --__next;
1799     while (__comp(__val, __next))
1800     {
1801         *__last = _GLIBCXX_MOVE(*__next);
1802         __last = __next;
1803         --__next;
1804     }
1805     *__last = _GLIBCXX_MOVE(__val);
1806 }
```

- Путь вызова: `__unguarded_linear_insert` ← `__unguarded_insertion_sort` ← `__final_insertion_sort`. То есть этот код выполняется один раз в самом конце, а не внутри рекурсии.

Почему цикл не убежит за начало массива

- Первые 16 элементов уже отсортированы «с проверкой», значит $a[0]$ — минимум этих шестнадцати.
- Разбиение Хоара никогда не переносит элемент через опорный: слева от любой позиции i лежит хотя бы один элемент, не больший $a[i]$.
- Этот элемент и есть сторож: сравнение `comp(val, next)` рано или поздно даст «ложь» и цикл остановится.

Минус одна проверка в самом горячем цикле библиотеки.
Насколько это выгодно — измерим в блоке 2.

Итог блока 1

- Сортировка — не «пройденный этап»: она внутри поиска, строк, СУБД и генерации текста нейросетью.
- Единица стоимости бывает разной: сравнение, обращение к диску, джоуль.
- `std::sort` — это `introsort`: медиана трёх, разбиение без проверок, порог 16, один финальный проход вставками, `heapsort` как страховка.

Дальше: и в этом вылизанном коде остаётся простор в десятки процентов.

Блок 2: число операций — это не время

Каждый эксперимент дальше устроен одинаково:
гипотеза → счётчики → время → сюрприз.

Источник: Andrei Alexandrescu, «Sorting Algorithms: Speed Is Found In The Minds of People», CppCon 2019 — и наши повторы его опытов на стенде.

слайды Александреску — слайды доклада; [demo/bench-alexandrescu/results.md](#) — [demo/bench-alexandrescu/results.md](#).

Сколько стоит сортировка вставками

- Худший случай: $n(n - 1)/2$ сравнений и столько же перемещений.
- Средний: $n(n - 1)/4$. При $n = 32$ это **248**.
- Именно ей библиотека дочищает короткие куски — значит, эти 248 операций выполняются миллионы раз.

Почему $n = 32$, а не 16: 32 — порог, с которым работает Алехсандреску (значение по умолчанию в реализации Microsoft), и вся его работа про то, чтобы порог *увеличить*. Поэтому дальше и у него, и у нас размер малого массива — параметр, от 8 до 256.

Очевидное улучшение: искать место бинарным поиском

- Место вставки i -го элемента ищем не линейно, а бинарным поиском: $C(n) = \sum_{i=1}^{n-1} \lceil \log_2 i \rceil$.
- $C(32) = \mathbf{155}$ против 248 — сравнений заметно меньше.
- Перемещений столько же: хвост всё равно надо сдвинуть.

Меньше работы за тот же результат $\Rightarrow$ выигрыш?

Оба варианта — наш код

```
54 -
55 void sort_guarded(T* a, size_t n, Cmp lt = Cmp()) {
56     for (size_t i = 1; i < n; ++i) {
57         T x = a[i];
58         size_t j = i;
59         while (j > 0 && lt(x, a[j - 1])) { a[j] = a[j - 1]; --j; }
60         a[j] = x;
61     }
62 }

90 -
91 void sort_binary(T* a, size_t n, Cmp lt = Cmp()) {
92     for (size_t i = 1; i < n; ++i) {
93         T x = a[i];
94         T* pos = std::upper_bound(a, a + i, x, lt);
95         std::move_backward(pos, a + i, a + i + 1);
96         *pos = x;
97     }
```

demo/bench-alexandrescu/insertion.cpp, строки 54-61 и 90-96.

Александреску: и что вышло по времени

1 млн случайных double, порог 32	std::sort	+ binary insertion
сравнений	25,33 млн	22,14 млн (−15 %)
перемещений	13,79 млн	13,79 млн (одинаково)
время	60,75 мс	68,58 мс (+13 %)

Меньше сравнений, столько же перемещений — и на 13 % дольше.

слайды Александреску, слайды 14–15; железо на слайдах не указано.

Платим за непредсказуемость переходов

- Процессор угадывает исход `if` заранее и работает наперёд; ошибка — сброс конвейера.
- Линейный поиск предсказуем: цикл почти всегда продолжается, и ошибается предсказатель примерно один раз на поиск. $R(32) = 87,5\%$ угаданных.
- Бинарный поиск на каждом сравнении извлекает ровно один бит информации — угадать нельзя, 50 %.

uint32, на ключ		guarded	binary	std::sort
сравнений	$n = 32$	8,62	3,73	5,94
	$n = 256$	64,73	6,63	9,53
ошибок предсказания	$n = 32$	1,044	2,116	—
	$n = 256$	1,111	4,636	3,522
время, нс	$n = 32$	7,55	16,89	11,20
	$n = 256$	20,68	29,66	19,14

Сравнений у `binary` в **10 раз** меньше — а времени в **1,43** раза больше. И `guarded` при $n = 256$ делает 64,73 сравнения на ключ против 9,53 у `std::sort`, почти всемеро больше, — а по времени это 20,68 нс против 19,14, разница **8 %**.

Замер вживую — `demo/live/cmp-vs-time.sh`, меньше 30 секунд.

стенд: Ryzen AI MAX+ 395, g++ 13.3, -O2, ключи `uint32`; `demo/bench-alexandrescu/results.md`, эксперимент А и

«Аппаратные счётчики».

И симулятор говорит то же самое

ошибок предсказания на ключ, $n = 32$	guarded	binary	std::sort
железо, perf stat (Zen 5)	1,044	2,116	—
симуляция, valgrind	1,067	3,699	2,130

Гипотезу Александреску мы не приняли на слово, а проверили двумя независимыми инструментами — и они согласны.

стенд, perf stat, Zen 5, за вычетом baseline; valgrind — valgrind --branch-sim=yes.

Не путать две постановки

- У Александреску `binary insertion` стоит *внутри* `std::sort` как `small_sort` с порогом 32 — и даёт +13% ко времени всей сортировки миллиона элементов.
- У нас это чистая сортировка массива длины n — и разница в разы.
- Общее — знак эффекта, а не его величина.

слайды Александреску, слайд 15; `demo/bench-alexandrescu/results.md`, «Отношения».

Второй опыт: убрать проверку границы

```
65 -
66 void min_to_front(T* a, size_t n, Cmp lt) {
67     size_t m = 0;
68     for (size_t i = 1; i < n; ++i)
69         if (lt(a[i], a[m])) m = i;
70     std::swap(a[0], a[m]);
71 }

76 -
77 void sort_unguarded(T* a, size_t n, Cmp lt = Cmp()) {
78     if (n < 2) return;
79     min_to_front(a, n, lt);
80     for (size_t i = 2; i < n; ++i) {
81         T x = a[i];
82         size_t j = i;
83         while (lt(x, a[j - 1])) { a[j] = a[j - 1]; --j; } // no j > 0 test
84         a[j] = x;
85     }
86 }
```

demo/bench-alexandrescu/insertion.cpp, строки 65-70 и 76-85.

Проверок меньше — а время?

uint32, $n = 32$, на ключ	guarded	unguarded	
переходов	18,678	13,870	−26 %
инструкций	75,238	63,865	−15 %
тактов	40,548	44,951	+11 %
время, нс	7,55	8,18	+8 %

Минус четверть переходов, минус шестая часть инструкций — и медленнее. Платим за лишний проход, поиск минимума: это $n - 1 = 31$ сравнение против 248 в среднем у самой сортировки, примерно **12 %** всей работы. А убранная проверка почти ничего и не стоила — она предсказуема. При $n = 256$ лишний проход относительно дешевле, и знак меняется: следующий кадр.

стенд, perf stat, за вычетом baseline. Проценты — отношение соседних столбцов; 12 % = 31/248 из A2.

При $n = 256$ знак меняется

uint32, $n = 256$, на ключ	guarded	unguarded	
переходов	129,617	68,402	-47 %
инструкций	514,307	347,070	-33 %
тактов	110,063	103,321	-6 %
время, нс	20,68	19,42	-6 %

Мораль: убрали половину переходов — получили 6 %.
Оптимизация есть, но она не там, где счётчик.

стенд, perf stat, за вычетом baseline.

Третий опыт: где обрывать quicksort

- В `libstdc++` порог = 16: куски короче не разбиваются, их дочищают вставки.
- Почему 16? Возьмём собственный quicksort и переберём порог C .
- 10^6 ключей `uint32`, медиана трёх, разбиение Хоара, листья дочищаются guarded-вставками.

порог 32 в VS, 16 в `libstdc++`, 30 (или 6) в `libc++`; код — `insertion.cpp`, эксперимент В.

Плато, а не точка

порог C	0	8	16	32	64	128	256
uint32, мс	55,50	52,69	49,18	45,77	44,68	44,16	46,36
double, мс	67,08	63,70	59,27	55,47	53,60	53,16	54,85

`std::sort` на тех же данных — **44,63** мс на `uint32` и **55,82** мс на `double`.

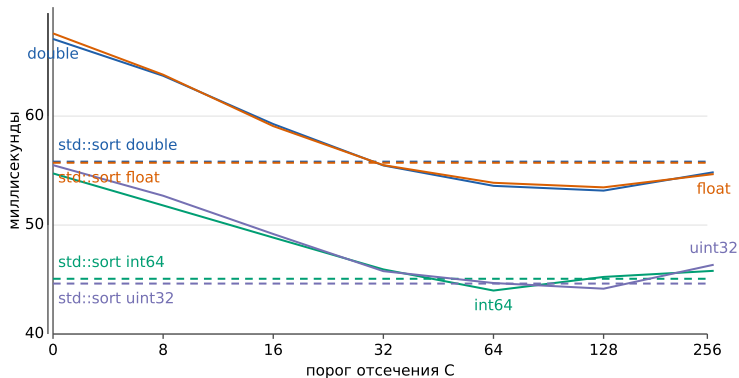
Ошибок предсказания на ключ: 18,080 при $C = 0$ против 15,505 при $C = 128$.

стенд, $n = 10^6$, `demo/bench-alexandrescu/results.md`, эксперимент В и «Part 1b».

Демо: время как функция порога

демо

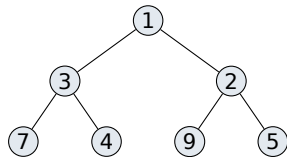
quicksort с отсечением на вставки: время от порога C , $n = 10^6$



Интерактивно — тетрадь `magimo`: время как функция порога, все четыре типа ключа. Адрес — на доске.

Напоминание: куча

- Массив, в котором для каждого i выполнено $a[i] \leq a[2i + 1]$ и $a[i] \leq a[2i + 2]$ (min-куча).
- `make_hear` расставляет элементы за $O(n)$.
- Нужно одно следствие: минимум оказывается в $a[0]$, а всё остальное — «примерно на своих местах».



$a = 1\ 3\ 2\ 7\ 4\ 9\ 5$

Подробно — в дне про деревья.

Используется в `std::sort` как страховка (`__partial_sort`).

Четвёртый опыт: сначала куча, потом вставки

- Идея Александреску: `make_heap`, затем вставки без проверки границы, начиная с `begin + 2`. «Like Smoothsort, just stupid».
- Счётчики: сравнений 23,51 млн (−9%), перемещений 11,56 млн (−20%) против 25,33 / 13,79 млн у `std::sort`.
- Время: 60,54 → **65,92** мс (+9%). Его слово — «disaster».

слайды Александреску, слайды 27–32.

Дальше он дрался за каждый такт

- Микрооптимизация `heapify` (бесконечные циклы вместо условий): 61,85 мс — всё ещё +2 % к исходным 60,54.
- Порог 63 вместо 32: первый выигрыш — −3 % по времени на `double`.
- Порог **255**: сравнений +46 %, перемещений +88 % — и время **57 мс**, то есть −4 %.

Объяснение у него — метрика $D(n)$: среднее расстояние между соседними обращениями к памяти. Выигрывает тот, кто ходит по памяти ближе.

Мы повторили — и получили не то же самое

нс на ключ	heap + unguarded	std::sort	
uint32, $n = 256$	18,36	19,14	−4,1 %
double, $n = 256$	20,98	23,54	−10,9 %
uint32, $n = 32$	12,11	11,20	+8 %

При $n = 32$ куча проигрывает и `std::sort`, и простым вставкам (7,55 нс). Выигрыш появляется только к $n = 256$.

стенд, `demo/bench-alexandrescu/results.md`, эксперимент A.

Пятый опыт: а если вход не случайный?

- **sorted** — уже отсортирован; **reversed** — отсортирован наоборот;
- **organpipe** — вверх до середины и вниз;
- **rotated** — $1, 2, \dots, n - 1, 0$: почти отсортирован, но один элемент не на месте;
- **random01** — случайные нули и единицы.

Его числа по формам входа

1 млн double, миллисекунды	plain insertion	heap insertion
sorted	10,92	9,99
reversed	8,43	8,13
organpipe	47,28	49,47
random01	14,27	11,45
rotated	обе версии падают в heapsort	

Про rotated он говорит: выбор опорного в GNU даёт квадратичное время — «у команды libstdc++ есть домашнее задание».

Домашнее задание сделано: наш замер

10 ⁶ ключей, мс	random	sorted	reversed	organpipe	rotated	random01
double	59,2	6,0	4,5	50,2	44,4	9,3
uint32	48,6	5,6	3,7	45,1	38,7	7,2

Rotated *быстрее* случайного входа. Квадратичного поведения нет.

стенд: g++ 13.3, -O2, taskset -c 1, лучшее из 5 прогонов; out-rot.txt.

Что ещё лежит в стандартных библиотеках

- **pdqsort** (pattern-defeating quicksort, Orson Peters) — в Go с версии 1.19 (`src/sort/sort.go:56`); в Rust `slice::sort_unstable` основан на `ipnsort` тех же авторов.
- **Powersort** — стратегия слияния серий в `list.sort()` CPython начиная с 3.11 (`Objects/listobject.c`, строка 1997).

То, что мы читали в блоке 1, — не единственный и не последний вариант.

Проверено по исходникам 11.09.2026; ссылки — [plan.md](#), § 2.7.

AlphaDev: сети сортировки, найденные обучением

- Mankowitz et al., «Faster sorting algorithms discovered using deep reinforcement learning», Nature 618 (2023).
- Из статьи: найденные решения для sort 3, sort 4 и sort 5 «have been integrated into the standard sort function in the LLVM standard C++ library».
- В libc++ 18 на стенде действительно лежат безветвенные `__sort3/4/5_maybe_branchless` поверх `__cond_swap`.

libc++, `__algorithm/sort.h` `__algorithm/sort.h`, строки 159-251. Соответствие именно этих строк статье — через ревью LLVM, нами не проверялось.

Выводы Александреску — и мостик в часть 3

- «Measure everything like crazy».
- «Code that wants to be fast is left-leaning» — то есть без глубоких ветвлений.
- Жёсткие сети сравнений для маленьких n .
- **Поразрядная сортировка для маленьких целых.**
- Порог выбирать по стоимости сравнения и перемещения и по `sizeof`, а не по традиции.

слайды Александреску, слайды 65–67.

<code>sort_guarded<uint32_t>, n = 256</code>	нс на ключ
цикл выровнен по 64 байтам (-falign-loops=64)	20,68
цикл пересекает 64-байтную границу	33,08

Разница в **1,60** раза. Инструкции — те же самые.

стенд с выравниванием кода против стенда без него без выравнивания кода, demo/bench-alexandrescu/results.md,

«Выравнивание кода».

- Вставлять не слева направо, а от середины наружу.
- Сравнений 23,75 млн (-7%), перемещений 12,15 млн (-13%).
- Время — то же в пределах 1% .

Ещё один способ сделать меньше работы и не выиграть ничего.

Блок 3: стена $n \log n$

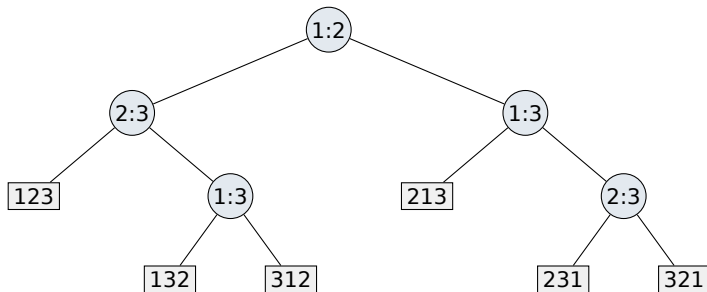
- Весь блок 2 — борьба за константу: пороги, сторожа, выравнивание.
- Сейчас докажем, что константой дело и ограничивается: *любая* сортировка сравнениями делает $\Omega(n \log n)$ сравнений.
- А потом обойдём эту оценку — не хитростью, а знанием о ключах.

Модель: дерево решений

- Зафиксируем алгоритм и размер входа n .
- Единственное, что алгоритм узнаёт о данных, — исходы сравнений $a_i : a_j$.
- Все возможные выполнения складываются в двоичное дерево: внутренние узлы — сравнения, листья — ответы (перестановки).
- Число сравнений в худшем случае — высота дерева h .

Дерево решений — не структура данных. Это описание всех выполнений алгоритма сразу.

Дерево для вставки трёх элементов



Шесть листьев — это $3! = 6$. Высота 3: худший случай — три сравнения.

Теорема о нижней оценке

- Каждая из $n!$ перестановок должна быть достижима — иначе на каком-то входе алгоритм выдаст неверный ответ. Значит листьев $l \geq n!$.
- Двоичное дерево высоты h имеет не больше 2^h листьев: $n! \leq l \leq 2^h$.
- Отсюда $h \geq \log_2(n!)$.
- Без формулы Стирлинга: $n! \geq (n/2)^{n/2}$, значит $h \geq \frac{n}{2} \log_2 \frac{n}{2} = \Omega(n \log n)$.

Что это значит для блока 2

Всё, что делали Александреску и мы, — борьба за константу.

$n \log n$ — это стена.

Обойти её можно только одним способом: знать о ключах больше, чем «их можно сравнить».

Сортировка подсчётом: идея

- Пусть все ключи — целые из $[0, k)$.
- Тогда не надо ничего сравнивать: достаточно посчитать, сколько раз встретился каждый ключ.
- Префиксные суммы счётчиков говорят, куда попадёт каждый элемент.
- Никакого дерева решений: алгоритм не сравнивает ключи вообще — значит нижняя оценка к нему не относится.

Весь алгоритм — семь строк

```
17 static void counting_sort(vector<u32>& a, u32 k) {
18     vector<u32> cnt(k, 0), out(a.size());
19     for (u32 x : a) cnt[x]++;
20     for (u32 i = 1; i < k; i++) cnt[i] += cnt[i - 1];
21     for (size_t i = a.size(); i-- > 0;) out[--cnt[a[i]]] = a[i];
22     a.swap(out);
23 }
```

- Три прохода: посчитать, накопить, разложить.
- Последний цикл идёт *с конца* — об этом через два кадра.

demo/bench/bench.cpp, строки 17-23.

Пример: $A = \langle 2, 5, 3, 0, 2, 3, 0, 3 \rangle$, $k = 6$

значение v	0	1	2	3	4	5
сколько раз встретилось	2	0	2	3	0	1
накопленная сумма $C[v]$	2	2	4	7	7	8

$C[v]$ — это номер последней позиции, куда может попасть значение v .

Например, все тройки займут позиции с 5-й по 7-ю.

В массиве восемь элементов, а счётчиков шесть: k — число различных значений, а не длина массива.

Раскладка — справа налево

берём	$A[i]$	кладём в	становится C
$i = 8$	3	$B[7]$	$C[3] = 6$
$i = 7$	0	$B[2]$	$C[0] = 1$
$i = 6$	3	$B[6]$	$C[3] = 5$
$i = 5$	2	$B[4]$	$C[2] = 3$
...			

Результат: $B = \langle 0, 0, 2, 2, 3, 3, 3, 5 \rangle$.

Почему с конца, и почему это важно

- Идя с конца, мы кладём *последнее* вхождение значения на последнюю свободную для него позицию — порядок одинаковых ключей сохраняется.
- Такая сортировка называется **устойчивой**.
- Если бы у ключа были спутные данные (номер, строка), порядок был бы виден — и порядок был бы нарушен.
- Устойчивость понадобится уже через три кадра: без неё поразрядная сортировка не работает вообще.

Свойства и цена

- Время $\Theta(n + k)$, память $n + k$.
- Устойчива.
- Хороша, когда $k = O(n)$. Это *условие*, а не факт.
- Ключи должны быть целыми и небольшими — по сути это адреса ячеек.

Наши замеры: $n = 10^6$, ключи меньше 2^{16}

мс	время	во сколько раз быстрее
std::sort	41,8	—
подсчёт	1,4	29,86
подсчёт без спутных данных	0,9	46,44

Это не «на несколько процентов». Это другой порядок.

стенд: g++ 13.3, -O2, taskset -c 1; demo/bench/results.md, demo/bench/results.md.

А теперь ключи меньше 2^{24}

$n = 10^6$, ключи меньше 2^{24}	время, мс	во сколько раз быстрее
<code>std::sort</code>	45,2	—
подсчёт	24,7	1,83

При $k = 2^{16}$ подсчёт был быстрее в 29,86 раза — теперь в 1,83. Алгоритм тот же, данные те же, изменился только диапазон ключей.

Прوماхов кэша на ключ у подсчёта: 0,980 при $k = 2^{16}$ против **3,169** при $k = 2^{24}$. Массив из 16 млн счётчиков в кэш не помещается.

стенд, perf stat, за вычетом baseline; demo/bench/results.md.

вход A	2	5	3	0	2	3	0	3
результат B	0	0	2	2	3	3	3	5

Тот же пример целиком — в ноутбуке `marimo`: видны массив C , накопленные суммы и каждый шаг раскладки справа налево. Адрес — на доске.

CLRS § 8.2; ноутбук — `demo/radix_sort.py` (EDUC-8), тесты — `demo/test_radix_sort.py`.

Поразрядная сортировка: пример

вход	по единицам	по десяткам	по сотням
329	72 <u>0</u>	7 <u>2</u> 0	<u>3</u> 29
457	35 <u>5</u>	3 <u>2</u> 9	<u>3</u> 55
657	43 <u>6</u>	4 <u>3</u> 6	<u>4</u> 36
839	45 <u>7</u>	8 <u>3</u> 9	<u>4</u> 57
436	65 <u>7</u>	3 <u>5</u> 5	<u>6</u> 57
720	32 <u>9</u>	4 <u>5</u> 7	<u>7</u> 20
355	83 <u>9</u>	6 <u>5</u> 7	<u>8</u> 39

Три прохода устойчивой сортировки подсчётом — по одной цифре, начиная с младшей.

- Тот же пример 329/457/657/839/436/720/355 в ноутбуке `marimo`: каждый проход — это сортировка подсчётом по одной цифре, видно счётчики и накопленные суммы.
- Число бит на цифру — ползунком. Для 32-битных ключей 1 бит — это 32 прохода по 2 счётчика, 16 бит — 2 прохода по 65 536.
- Адрес ноутбука — на доске.

Чем толще цифра, тем меньше проходов — и тем больше счётчиков. Чем это кончается, измерим через несколько кадров.

Ноутбук — `demo/radix_sort.py` (EDUC-8); тесты — `demo/test_radix_sort.py`. Числа проходов и счётчиков — арифметика:

$\lceil 32/r \rceil$ и 2^r .

LSD: алгоритм и теорема

- Ключ — d цифр, каждая из алфавита размера k .
- Для $i = 1..d$: устойчиво отсортировать весь массив по i -й цифре (младшая — первая).
- Время $\Theta(d(n + k))$, если внутренняя сортировка устойчива и работает за $\Theta(n + k)$.
- «Цифра» — это что угодно: байт, поле даты, символ строки.

Без устойчивости не работает

- Пусть после прохода по единицам стоит $\langle 720, 329 \rangle$ — порядок по младшей цифре уже учтён.
- Проход по десяткам: у обоих цифра 2. Неустойчивая сортировка вправе поменять их местами — и результат младшего прохода потерян.
- Индукция: после i -го прохода массив отсортирован по последним i цифрам. Шаг индукции опирается ровно на устойчивость.

Наша реализация: подготовка

```
33  const int D = (32 + R - 1) / R, K = 1 << R;  
34  vector<u32> buf(a.size());  
35  vector<size_t> hist((size_t)D * K, 0);  
36  for (u32 x : a) for (int d = 0; d < D; d++) hist[(size_t)d * K + ((x >> (d * R)) & (K - 1))]++;  
37  u32* src = a.data(); u32* dst = buf.data(); size_t n = a.size();
```

- R бит на цифру, $D = \lceil 32/R \rceil$ проходов, $K = 2^R$ счётчиков на проход.
- Гистограммы всех цифр считаются за один проход по данным.

demo/bench/bench.cpp, строки 33-37.

Наша реализация: один проход

```
38  for (int d = 0; d < D; d++) {
39      size_t* h = &hist[(size_t)d * K];
40      if (h[(src[0] >> (d * R)) & (K - 1)] == n) continue; // all keys share this digit
41      size_t sum = 0;
42      for (int i = 0; i < K; i++) { size_t c = h[i]; h[i] = sum; sum += c; }
43      for (size_t i = 0; i < n; i++) dst[h[(src[i] >> (d * R)) & (K - 1)]++] = src[i];
44      swap(src, dst);
45  }
46  if (src != a.data()) memcpy(a.data(), src, n * sizeof(u32));
```

- Строка 40 — если все ключи имеют одну и ту же цифру, проход пропускается.

demo/bench/bench.cpp, строки 38–46.

Сколько бит брать на цифру: теория

- Ключ из b бит, цифра из r бит: $\lceil b/r \rceil$ проходов, в каждом $n + 2^r$ работы.
- Итого $\Theta(\frac{b}{r} (n + 2^r))$.
- При $b \geq \log_2 n$ минимум достигается около $r = \lfloor \log_2 n \rfloor$.
- Для $n = 10^7$ это $r \approx 23$: «берите цифру пожирнее, проходов будет меньше».

Сколько бит брать на цифру: измерение

$n = 10^7$, 32-битные ключи	время, мс	тактов/ключ	промахов L1d
radix8 — 4 прохода по 256 счётчиков	47,3	29,9	7 536 044
radix11 — 3 прохода по 2048 счётчиков	73,9	45,3	28 436 566

Проходов меньше — времени больше в 1,56 раза. Теорема не считает кэш.

стенд, -02, perf stat; demo/bench/results.md.

Поразрядная против `std::sort`

$n = 10^7, k = 2^{16}$	тактов на ключ	промахов кэша на ключ
<code>std::sort</code>	211,1	7,288
поразрядная (8 бит)	23,5	0,026

Отношение тактов: **9,0 / 10,9 / 9,2** для $k = 2^{16}/2^{24}/2^{32}$.

стенд, -O2, perf stat, за вычетом baseline; demo/bench/results.md.

Подсчёт против поразрядной

$n = 10^6, k = 2^{24}$	время, мс	тактов на ключ	промахов кэша на ключ
подсчёт	24,7	81,5	3,169
поразрядная (8 бит)	3,0	25,3	0,176

Обе линейные, обе на одних и тех же данных. Разница по времени в 8 раз — это разница по промахам кэша в 18 раз: подсчёт ходит по массиву из 16 млн счётчиков, поразрядная — по 256.

стенд, -02, perf stat, за вычетом baseline; demo/bench/results.md.

Два возражения — и ответы

- «Это оптимизатор развернул циклы». На -O0 отношение **13,61-15,92** — разрыв только растёт.
- «На больших данных сравнивается». $n = 10^8$, $k = 2^{16}$:
std::sort **4011,7** мс, поразрядная **360,0** мс.
- Цена: лишний массив на n элементов и требование «ключ — целое фиксированной ширины».

стенд, demo/bench/results.md, «Отношения» (строки O0) и «Масштабирование».

Карманная сортировка: ещё одно предположение о ключах

- Пусть ключи — вещественные числа, равномерно распределённые в $[0, 1)$.
- Заведём n «карманов»: элемент a кладём в карман $\lfloor n \cdot a \rfloor$.
- Каждый карман сортируем вставками, потом выписываем карманы подряд.

Предположение здесь не про *тип* ключа, а про его *распределение*.

Пример: 10 чисел

карман	содержимое после раскладки		
1	0,17	0,12	
2	0,26	0,21	0,23
3	0,39		
6	0,68		
7	0,78	0,72	
9	0,94		

Вход: 0,78 0,17 0,39 0,26 0,72 0,94 0,21 0,12 0,23 0,68.

Карманы 0, 4, 5, 8 пусты. Осталось отсортировать каждый карман и склеить.

Корректность и худший случай

- Корректность: если $a_i \leq a_j$, то $\lfloor na_i \rfloor \leq \lfloor na_j \rfloor$ — элемент не может попасть в карман правее «своего места».
- Значит, склейка отсортированных карманов отсортирована.
- Худший случай: все n элементов в одном кармане, вставки дают $\Theta(n^2)$.

Вопрос: почему тогда её вообще используют?

Два определения (только для этого доказательства)

- **Матожидание** $E[X]$ — среднее значение случайной величины по её распределению.
- **Индикатор** X — величина, равная 1, если событие произошло, и 0 иначе. Для неё $E[X] = \Pr\{X = 1\}$ и $X^2 = X$.
- **Линейность**: $E[X + Y] = E[X] + E[Y]$ всегда, даже если X и Y зависимы. Принимаем без доказательства.

Доказательство: сколько стоит один карман

- $X_{ij} = \mathbb{I}\{A[j] \text{ попал в карман } B[i]\}$, тогда $n_i = \sum_j X_{ij}$ — размер кармана i .
- Вставки внутри кармана стоят $O(n_i^2)$, значит нужно $E[n_i^2]$.

•

$$E[n_i^2] = \sum_j E[X_{ij}^2] + \sum_{j \neq k} E[X_{ij}X_{ik}]$$

- Первая сумма: $X^2 = X$, вероятность попасть в конкретный карман — $1/n$, слагаемых n .
- Вторая: ключи независимы, поэтому $E[X_{ij}X_{ik}] = \frac{1}{n} \cdot \frac{1}{n}$, слагаемых $n(n-1)$.

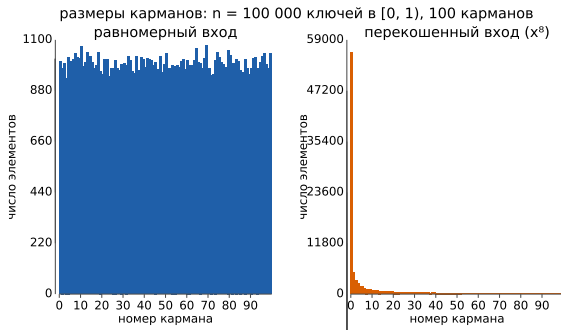
Доказательство: итог

$$E[n_i^2] = n \cdot \frac{1}{n} + n(n-1) \cdot \frac{1}{n^2} = 2 - \frac{1}{n} < 2$$

- Ожидаемая работа внутри одного кармана — константа.
- Суммарно: $\Theta(n) + \sum_{i=0}^{n-1} O(2 - 1/n) = \Theta(n)$.
- Сравнения внутри карманов есть — но их ожидаемое число линейно, потому что мы заранее знаем, как ключи распределены.

Что будет, если вход перекошен

демо



- 10^5 ключей, 100 карманов.
- Равномерный вход: самый большой карман **1076**, самый маленький **935**.
- Перекошенный (те же значения в восьмой степени): **56 137** против **119**.

Больше половины массива в одном кармане — и вставки в нём дают квадратичное время. Интерактивно — тетрадь `marimo`: перекос входа, n , p , m . Адрес — на доске.

Гистограмма размеров карманов, равномерный вход против перекошенного; данные — `demo/plots/bucket-data.txt`,

генератор — `demo/plots/bucket_sizes.py`.

Внешняя сортировка: другая единица стоимости

- Данные не помещаются в память. Считаем не сравнения, а обращения к диску.
- Одно чтение блока стоит как миллионы сравнений — поэтому алгоритм строится вокруг того, сколько раз данные прочитаны целиком.

Это возвращает нас к самому началу дня: петабайт, 48 тысяч дисков, шесть часов.

Серии и слияние

1. Читать по M записей (сколько влезает в память), сортировать в памяти, писать на диск как *серию*.
2. Сливать серии по k штук: на каждом шаге берём минимум из k текущих элементов — это куча размера k .
3. Повторять, пока не останется одна серия.

Число проходов по данным: $1 + \lceil \log_k(N/M) \rceil$.

Та же схема на кластере

- Tencent Sort, 2016: 100 ТБ за **98,8** с на **512** узлах.
- Это те же «отсортировать куски — слить», только куски лежат на разных машинах, а слияние идёт по сети.
- DuckDB на одной машине: поразрядная сортировка в каждом потоке, затем k -путевое слияние, с выгрузкой блоков на диск.

sortbenchmark.org, GraySort Indy 2016 (чужое железо); блог DuckDB, 27.08.2021.

Что делают сегодня (1): модели вместо сравнений

- **Learned Sort** (SIGMOD 2020): по выборке строится модель функции распределения, ключ сразу кладётся на предсказанную позицию. По сути карманная сортировка, где карманы выбирает обученная модель.
- Из аннотации продолжения: версия 2.0 быстрее первой в **4,78** раза на данных с повторами и в **1,60** — без.

По данным авторов, на их железе. Чисел исходной статьи мы проверить не смогли — страница ACM с этой машины не открывается.

Что делают сегодня (2): параллельно и на SIMD

- **IPS⁴o** — in-place samplesort: до **3×** против ближайшего in-place конкурента, последовательно до **1,5×** против BlockQuicksort.
- **NumPy 1.25** — `np.sort` на AVX-512 до **15×** (16-битные типы) и **9×** (64-битные), `np.argsort` до **6×**.
- **Onesweep** — поразрядная на GPU: **29,4** GKey/s на NVIDIA A100 для 256 млн 32-битных ключей, примерно **1,5×** против CUB.

Все три — по данным авторов, на их железе.

arXiv:1705.02257; заметки к выпуску NumPy 1.25 (gh-22315, gh-23707); arXiv:2206.01784.

$n = 10^7$, uint32	$k = 2^{32}$, мс	$k = 2^{16}$, мс
<code>std::sort</code>	522,4	406,6
поразрядная (8 бит)	47,3	36,4
подсчёт	—	41,9

Отношение времён при $k = 2^{32}$ — **11,0**. Замер вживую — `demo/live/radix-vs-std.sh`: 10^7 случайных ключей на одном ядре, меньше 20 секунд. Это ровно тот baseline, против которого будет считаться соревнование.

стенд: `g++ 13.3, -O2, taskset -c 1; demo/bench/results.md`; эталонный вывод — `demo/live/expected/radix-vs-std.txt`.

Соревнование: задача

- Отсортировать большой массив 32-битных ключей быстрее, чем `std::sort`.
- Сдаётся одна программа. Вход и выход — файлы.
- Судья один и тот же для всех, запуск на одной машине.
- Наборов данных несколько, с разными распределениями — в том числе те самые формы входа из блока 2.
- Результат по набору — во сколько раз быстрее базовой сортировки.

Точные параметры — в правилах соревнования, отдельным объявлением.

Соревнование: таблица и призы

- Таблица результатов открыта, отправлять можно сколько угодно раз; адрес — на доске и в чате курса.
- **Первое место — экзамен «отлично» автоматом.**
- Призы поменьше — освобождение от одной лабораторной или одного домашнего задания.
- Пороги и сроки — отдельным объявлением.

Объявлено Андреем Калининым 11.09.2026.

Карта идей на сегодня

- Baseline — `std::sort`. Всё, что было сегодня, — это карта: порог отсечения, сторож, выравнивание, формы входа.
- Ключи — `uint32`. Честная поразрядная даёт больше 3 на равномерных наборах: это уже проверено на стенде.
- Но наборов девять, и на `sorted` или `fewdistinct` выигрывает другое.

`demo/bench/results.md` — та же машина, что и судья.

Попробуйте сами

- Соберите `rot.cpp` и `insertion.cpp` из материалов курса на своей машине и посмотрите, совпадут ли числа. У другого процессора и другого компилятора они совпасть не обязаны — это и есть самое интересное.
- Откройте заголовок своей стандартной библиотеки и найдите в нём порог и финальный проход вставками. У MSVC порог 32, у `libc++` — 30 или 6, смотря какой тип.
- Поиграйте с ноутбуком: поменяйте число бит на цифру и посмотрите, как меняется число проходов и счётчиков.
- Посмотрите доклад целиком:
<https://www.youtube.com/watch?v=FJJTYQYB1JQ>

Что читать

- CLRS, глава 8 «Сортировка за линейное время» — §§ 8.1–8.4 и приложение C.3.
- Кнут, том 3: 5.3.1 (нижняя оценка), 5.4 (внешняя сортировка).
- Andrei Alexandrescu, «Sorting Algorithms: Speed Is Found In The Minds of People», CppCon 2019: слайды в архиве курса, запись — <https://www.youtube.com/watch?v=FJJTYQYB1JQ>
- Наши таблицы замеров:
demo/bench-alexandrescu/results.md и
demo/bench/results.md — читать как образец того, как оформляется измерение.

Итог дня

- Сортировка — внутри поиска, строк, СУБД и генерации текста.
- `std::sort` — introsort: медиана трёх, порог 16, финальные вставки, heapsort как страховка.
- Меньше операций $\neq$ быстрее: мешают предсказание переходов и кэш. Мы видели это четыре раза.
- $\Omega(n \log n)$ — стена для сортировок сравнением.
- Выйти за неё можно только знанием о ключах: их множество (подсчёт, поразрядная) или их распределение (карманная).

Мерить. Всегда мерить.